Computer Algorithms

Dr. Ashish Seth

What is a Computer Algorithm?

- A computer algorithm is
  - **a detailed step-by-step method for**
  - **solving a problem**
  - **by using a computer.**

Properties

- Finiteness
- Unambiguous
- Definiteness of sequence
- Input /Output defined
- Feasibility

Design Techniques -Problem Solving Strategies

- Divide & Conquer
- Greedy Method
- Branch & Bound
- Backtracking
- Dynamic Programming
- Brute Force
- Randomized

Problem Solving Using Computers

- Problem:
- Strategy:
- Algorithm:
  - **Input:**
  - **Output:**
  - **Step:**
- Analysis:
  - **Time & Space:**
  - **Optimality:**
  - **Correctness:**
- Implementation:
- Verification:

Example: Search in an unordered array

- ***Problem***:
  - Let E be an array containing n entries, E[0], …, E[n-1], in no particular order.
  - Find an index of a specified key K, if K is in the array;
  - return –1 as the answer if K is not in the array.

- ***Strategy***:
  - Compare K to each entry in turn until a match is found or the array is exhausted.
  - If K is not in the array, the algorithm returns –1 as its answer.

Example: Sequential Search, Unordered

- ***Algorithm*** (and data structure)

  - **Input:**

    E, n, K, where E is an array with n entries (indexed 0, …, n- 1), and K is the item sought.

  - **Output:**

    Returns ans, the location of K in E (-1 if K is not found)

Algorithm: Step (Specification)

```
int seqSearch(int[] E, int n, int K)
int ans, index;
ans = -1;              // Assume failure.
for (index = 0; index < n; index++)
  if (K == E[index])
      ans = index;     // Success!
      break;           // Done!
return ans;
```

Analysis of the Algorithm

we measure the amount of work done by an algorithm.

- **Basic Operation:**
  - Comparison of x with an array entry

- **Worst-Case Analysis**:
  - Let W(n) be a function.

  W(n) is the maximum number of basic operations performed by the algorithm on any input size n.
  - For our example, clearly W(n) = n.

  - The worst cases occur when K appears only in the last position in the array OR when K is not in the array at all.

More Analysis of the Algorithm

- ***Optimality***:
  - The Best possible solution?
  - Searching an Ordered Array
    Using Binary Search
  - W(n) = Ceiling[lg(n+1)]
  - **The Binary Search algorithm is optimal.**

$$= \lceil \lg(n+1) \rceil$$

***Correctness* can be proved!**

Correctness: (proving correctness of procedures)

An algorithm consists of sequences of steps (operations, instructions, statements) for *transforming* inputs (***preconditions***) to outputs (***postconditions***)

- Proving
  - **if the preconditions are satisfied,**
  - **then the postconditions will be true,**
  - **when the algorithm terminates.**

Algorithm Language (Specifying the Steps)

- Java/C++ as an algorithm language can be used
- steps within an algorithm may be specified in pseduocode (English phrases)
- Focus on the strategy and techniques of an algorithm, not on detail implementation



Loop Invariant – Analysing Correctness of Algorithm

A tool used to prove the correctness of an algorithm, specifically in regards to a loop

It is a property or set of properties of type boolean and must be aligned with the goal of the algorithm

Must hold (be true) at initialization, maintenance, and termination

To prove correctness with a loop invariant we need to show three things:

Initialization

Invariant is true prior to the first iteration of the loop.

Maintenance

If the invariant is true before an iteration of the loop,
it remains true before the next iteration.

Termination

When the loop terminates,
the loop invariant along with the reason that the loop terminated
gives us a useful property.

Finding Max value in an array

```
def get_max(a):
  max = a[0]

  for i in range(1,len(a)):
    if a[i] > max:
      max = a[i]

  return max
```

Loop invariant:

- max is the maximum value in a[0..i-1]

Initialization:

- i=1 and the scope of the array is a[0..i-1]=a[0..1-1]=a[0], max=a[0]
- Thus, invariant holds

… finding max value in an array

```
def get_max(a):
  max = a[0]

  for i in range(1,len(a)):
    if a[i] > max:
      max = a[i]

  return max
```

Maintenance:

- Assuming loop invariant holds at iteration k, then max is the maximum value in a[0..k-1]
- In the next iteration k+1, the code compares and possibly updates max with a[k]. max is the maximum value in scope a[0..k]
- Thus, invariant holds

Termination:

- i = len(a), max is the maximum value in a[0..len(a)-1]
- Thus, invariant holds

Linear Search

Input:

- *A: an array*
- *n: the number of elements in A*
- *x: the value to be searched for*

Output: Either an index i for which A[i] = x,
or Not-Found if A does not contain x

1. For i = 0 to n-1:
   A. If A[i] = x, then return i
2. Return Not-Found

output is correct:

1. if index i is returned then A[i] = x
2. if Not-Found is returned then x is not in the array

1. if index i is returned then A[i] = x
2. if Not-Found is returned then x is not in the array

Loop invariant

At the start of each iteration of step 1, if x is present in the array A, then it is present in the subarray A[i:n]

Initialization

Initially, i=0 so that the subarray in the loop invariant is A[0:n], which is the entire array. Thus, the loop invariant holds initially.

Maintenance

If at the start of an iteration x is present in the array, then it is present the subarray A[i:n]. (means now you are at ith index and at this index you have find x)
If we do not return, then A[i] ≠ x.
Hence, if x is in the array then is it in the subarray A[i+1:n].
i is incremented before the next iteration. so the invariant will hold as

Termination

"if A then B" ≡ "if not B then not A"

If A[i] = x ✔. If i > n-1 consider contrapositive of invariant.

"if x is not present in the subarray A[i:n], then x is not present in A."

Amount of work done

- **We want a measure of work that tells us something about the *efficiency* of the method used by the algorithm**
- **independent of computer, programming language, programmer, and other implementation details.**
- **Usually depending on the *size of the input***

- Counting passes through loops

- Basic Operation
  - **Identify a particular operation fundamental to the problem**
  - **the total number of operations performed is roughly proportional to the number of basic operations**

- Identifying the properties of the inputs that affect the behavior of the algorithm

Worst-case complexity

- Let D_n be *the set of inputs of size n* for the problem under consideration, and let I be an element of D_n.
- Let t(I) be *the number of basic operations* performed by the algorithm on input I.
- We define the function W by

$$W(n) = \max\{t(I) \mid I \in D_n\}$$

- called the worst-case complexity of the algorithm.
- W(n) is the maximum number of basic operations performed by the algorithm on any input of size n.

- ***The input, I, for which an algorithm behaves worst depends on the particular algorithm***.

Space Usage

- worst-case analysis can be done.

==> **If memory cells used by the algorithms depends on the particular input**

➢Normally a program require space for instructions, constants, variables used by program and input data,

➢It may also use space for manipulating data and storing information to carry out computations

1. Fixed part: The size required to store certain data/variables, that is independent of the size of the problem:
   - **e.g. name of the data collection**
   - **same size for classifying 2GB or 1MB of texts**

1. Variable part: Space needed by variables, whose size is dependent on the size of the problem:
   - **e.g. actual text**
   - **load 2GB of text VS. load 1MB of text**

Generally we do
Time and Space Tradeoff.

Analysis Tool: Mathematics: Set



- A set is a collection of distinct elements.The elements are of the same "type",
- "element e is a member of set S" is denoted as e ∈ S
- A particular set is defined by listing or describing its elements between a pair of curly braces:
  $S_1 = \{a, b, c\}$, $S_2 = \{x \mid x \text{ is an integer power of } 2\}$
  read "*the set of* all elements x *such that* x is …"
- $S_3 = \{\} = \emptyset$, has not elements, called empty set
- A set has no inherent order.

Subset, Superset; Intersection, Union

- If all elements of one set, S_1
  - **are also in another set, S_2,**
- Then S_1 is said to be a *subset* of S_2, $S_1 \subseteq S_2$
  - **and S_2 is said to be a *superset* of S_1, $S_2 \supseteq S_1$.**
- Empty set is a subset of every set, $\emptyset \subseteq S$
- *Intersection*
- $S \cap T = \{x \mid x \in S \text{ and } x \in T\}$
- *Union*
- $S \cup T = \{x \mid x \in S \text{ or } x \in T\}$

Cardinality

- Cardinality
  - **A set, S, is *finite* if there is an integer n such that the elements of S can be placed in a one-to-one correspondence with {1, 2, 3, …, n}**
  - **in this case we write |S| = n**

- How many **distinct subsets does a finite set** on n elements have? There are **2^n subsets**.

- How many **distinct subsets of cardinality** k does a finite set of n elements have?
  There are C(n, k) = n!/((n-k)!k!), "n choose k"

Analysis Tool: Algebra



Manipulating Inequalities

- Transitivity: If ((A □ B) and (B □ C) Then (A □ C)
- Addition: If ((A □ B) and (C □ D) Then (A+C □ B+D)
- Positive Scaling:
  If ((A □ B) and (□ > 0) Then (□ A □ □ B)
- *Floor and Ceiling Functions*
- Floor[x] is the largest integer less than or equal to x.

ëxû

- Ceiling[x] is the smallest integer greater than or equal to x.

éxù

Logarithms



- For $b>1$ and $x>0$, $\log_b x = L$ such that $b^L = x$
- $\log_b$ is a strictly increasing function,
  if $x > y$ then $\log_b x > \log_b y$
- $\log_b$ is a one-to-one function,
  if $\log_b x = \log_b y$ then $x = y$
- $\log_b 1 = 0$; $\log_b b = 1$; $\log_b x^a = a \log_b x$
- $\log_b(xy) = \log_b x + \log_b y$
- $x^{\log y} = y^{\log x}$
- change base: $\log_c x = (\log_b x)/(\log_b c)$

Series



A *series* is the sum of a sequence.

- Arithmetic series
  - **The sum of consecutive integers**

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

- Polynomial Series
  - **The sum of squares**
  - **The general case is**

$$\sum_{i=1}^{n} i^2 = \frac{2n^3 + 3n^2 + n}{6} \gg \frac{n^3}{3}$$

$$\sum_{i=1}^{n} i^k \gg \frac{n^{k+1}}{k+1}$$

- Power of 2

$$\sum_{i=0}^{k} 2^i = 2^{k+1} - 1$$

- Arithmetic- Geometric Series

$$\sum_{i=1}^{k} i\,2^i = (k-1)2^{k+1} + 2$$

Optimality the best possible

- Each problem has inherent complexity
  - **There is some *minimum* amount of work required to solve it.**
- To analyze the complexity of a problem,
  - **we choose a class of algorithms (**by specifying the types of operations the algorithm is permitted to perform**), and**
  - **Measure of complexity (**ex, the basic operations to be counted**)**

An algorithm is optimal if there is no algorithm in the class under study that performs fewer operations (in worst case)

Optimal doesn't mean best known but it means best possible

Show whether an algorithm is optimal?

- Analyze the algorithm, call it A, and found the Worst-case complexity $W_A(n)$, for input of size n.
- Prove a theorem starting that,
  - **for any algorithm in the same class of A**
  - **for any input of size n, there is some input for which the algorithm must perform at least $W_{[A]}(n)$ (lower bound in the worst-case)**
- If $W_A(n) == W_{[A]}(n)$
  - **then the algorithm A is optimal**
  - **else there may be a better algorithm**
  - **OR there may be a better lower bound.**

Optimality e.g.

- Problem
  - **Fining the largest entry in an (unsorted) array of n numbers**
- Algorithm A
  - **int findMax(int[] E, int n)**
  - **1. int max;**
  - **2. max = E[0]; // Assume the first is max.**
  - **3. for (index = 1; index < n; index++)**
  - **4. if (max < E[index])**
  - **5. max = E[index];**
  - **6. return max;**

Analyze the algorithm, find $W_A(n)$

- Basic Operation
  - **Comparison of an array entry with another array entry or a stored variable.**
- Worst-Case Analysis
  - **For any input of size n, there are exactly n-1 basic operations**
  - **$W_A(n) = n-1$**

Video Lecture of the topic can be found at

Introduction to Computer Algorithms
https://youtu.be/GI1NwGPGxNo

Computer Algorithms - Abstract Data Types (ADT)

https://youtu.be/aHWNxF0c08U

Basics for Analysing Computer Algorithms

https://youtu.be/n1ArVTvMJtE